Deep Learning and GenAI Project Report

Term-1 2026

Predicting The Right Genre from Noisy Music Mashups

Name: Akash Kumbhakar

ID: 23F1001065@ds.study.iitm.ac.in

Indian Institute of Technology, Madras

BS in Data Science and Applications

Abstract:

This project is a Kaggle competition where each participant must make Artificial neural network based deep learning models for predicting the right genre of noisy music mashups. It is a multiclass classification problem in over 10 distinct music genres. I am provided with a curated training dataset consisting of songs from 10 distinct music genres, where each song is instrument-separated into four stems: **'drums.wav**', '**vocals.wav**', '**bass.wav**' and '**others.wav**'. There is also a dataset containing environmental noise sounds from 50 different environmental classes and a noisy test music mashup dataset for submission. As the model must predict genres of music with noisy background, this noise dataset will be helpful for training music mashups. The core challenge in this project lies in generalization. Instead of clean, original tracks, the test data is composed of **mashups** created by mixing instruments stems from different songs belonging to the same genre. To ensure musical coherence during mixing, some instrument tracks may undergo tempo adjustments so that all stems are rhythmically synchronized before being combined. To further increase complexity and simulate real-world audio conditions, **random noise samples** are added to these mashups at varying intensities and positions. Models are evaluated using the Macro F1 Score across the 10 distinct genre classes. Higher macro F1 scores indicate better overall genre classification performance across all classes and for submission a csv file of format **"id"** and **"genre"** must be submitted.

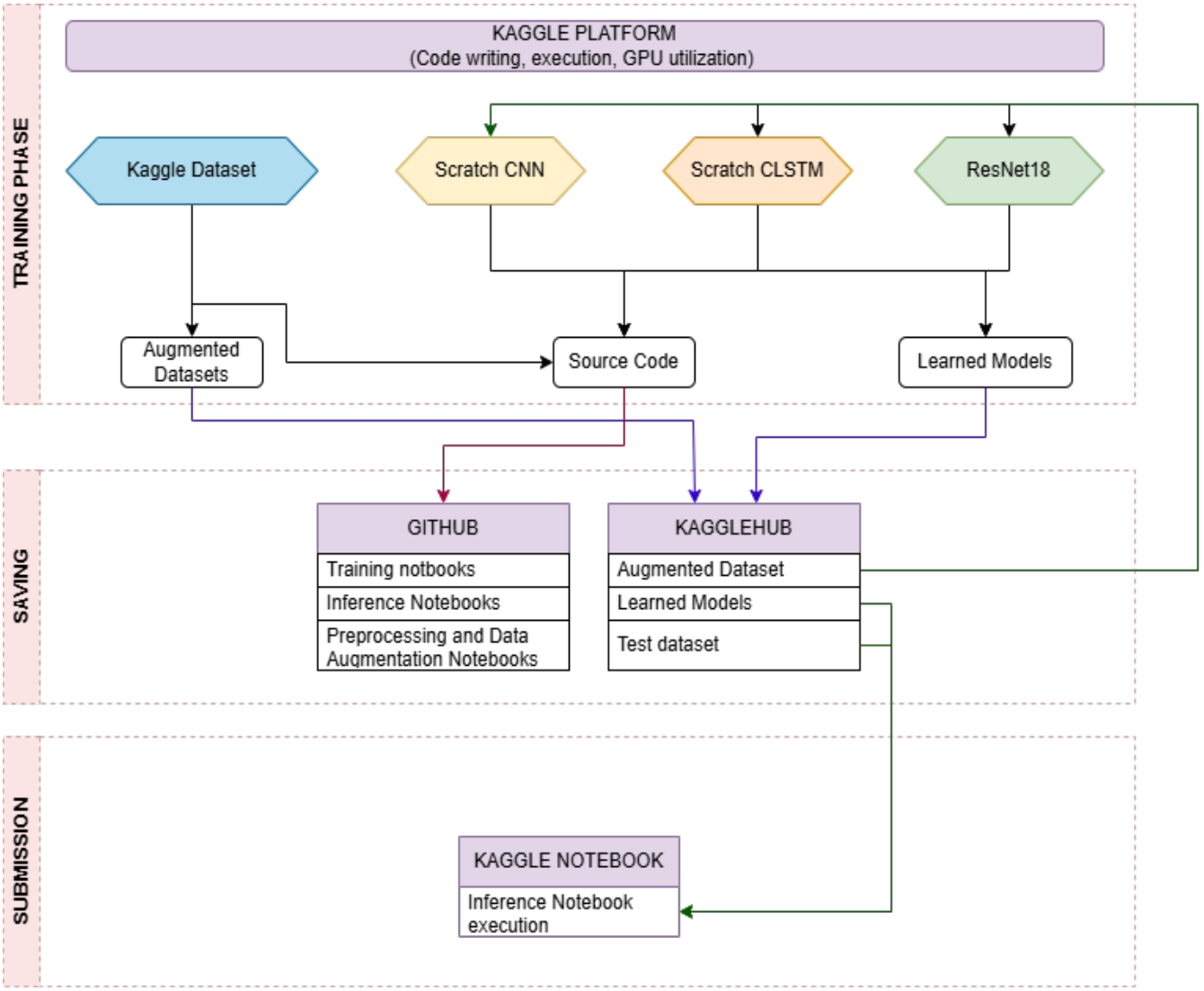
The competition provided a dataset of 1000 music, 100 from each genre. Each music is a folder that contains four instrument separated clean stems, combining these stems we get a clean full music of 30 seconds. There is also a set of 2000 environmental noise files of 5 seconds duration. As the actual test data is noisy and a combination of cross-song stems, a total of 150000 samples, 15000 from each genre with cross stems recombination within the genre, is created with added environmental noise at random position. Also, another two versions of datasets are created, one is 5000 samples for training, 500 for validation per genre with random gain at stem level, time stretch, time shift, pitch shift and with extra artificial noise in addition to environmental noise. Another one is 5000 pure samples for training and 500 for validation and added the previous version with it resulting in 10000 samples for training and 1000 for validation per genre. Three models of **Scratch CNN**, **scratch CLSTM**, and a pretrained **ResNet18** convolution model are employed. All models gave a test f1_score above 0.80 on the first dataset and the ResNet18 model finetuned giving best f1_score of 0.90.

The 150000(120000 train + 30000 validation) samples dataset perform well on these models. They have only environmental noise added at random positions and no other augmentation. Although during train validation set creation for this dataset, I have worked on all stems. Train validation separation at stem level is not done properly. After augmentation using all stems for each genre, I separated the train and validation set which ultimately leads to data leakage in my validation set. So, validation of metrics looks attractive during training, but that is misleading. But as that does not affect training the model, I keep them. After learning about this data leakage, I have created those two sets of datasets with proper stem level train-validation split. After all this I found good training dataset creation is the key for this noisy classification problem.

Introduction:

The main objective of this Kaggle competition is to build a robust music genre classification model using Artificial Neural Network based Deep Learning architecture. The classification model should be robust to correctly classify music with background noise and cross stems combination version of music. The music dataset is organized into clean instrument stems, environmental noise, and noisy mashup samples for test submission on Kaggle. My specific goal for this project is to build three models starting from scratch Convolutional Neural Network (CNN) and then the second is Convolutional Recursive Neural Network (CRNN) where I have used Long Short-Term Memory based sequence model after Convolution block to capture the temporal relationship of mashup music and then to utilize a pre-trained model and finetune it with my specific use case . Above all, the very important task that I must complete is to extract model feedable input features from provided raw music data. The useful features that I can get from each music is a music mel-spectrogram that contains all information on frequency domain, time domain and amplitude at each time point and frequencies present in the music. Mel-spectrograms are good input features as they capture temporal structure with frequency and amplitude value for the frequency that are present at a specific time moment in the music. As Mel-spectrograms are two-dimensional matrices, it also works as image-like data for our Convolution based model and as there is also temporal structure (axis) present in this input representation, this can be fed to sequence based models like RNN or LSTM. My first focus was to cross the cutoff f1_score set by the competition and then work on those models further to improve my score. I have learnt some tools from PyTorch, librosa, torchaudio library to work on this project and their links are given in the reference section of this report.

Project Flow :



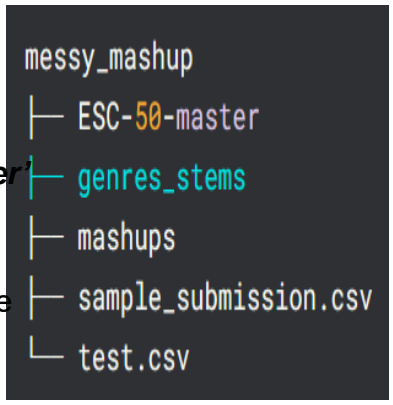
Report Structure:

- Abstract
- Introduction
- Project Flow
- Dataset and Preprocessing
- Mel-Spectrogram
- Dataset and Data Loader
- Modeling & Experimentation
    - CNN
    - CLSTM
    - ResNet18
- Performance and Comparative Analysis
- Comparative Table
- Conclusion

References

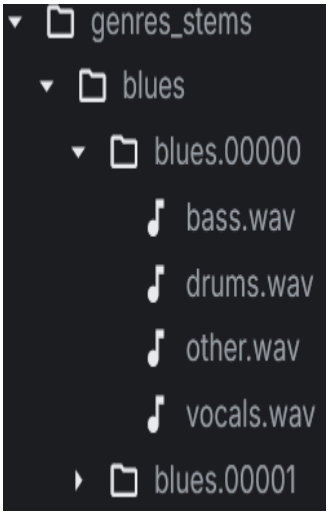
Dataset and Preprocessing:

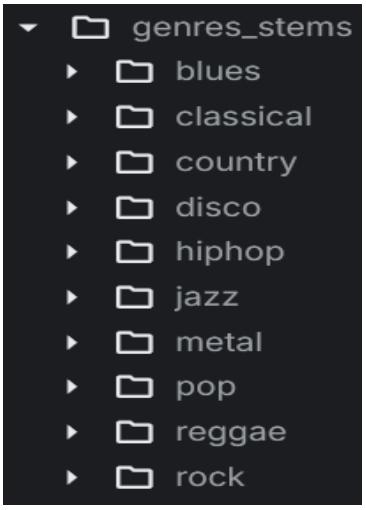
The Kaggle competition provided dataset are as follows--



'messy_mashup' is the main directory that contains all the data required in this project. This directory is organized into five folders where ***'ESC-50-master'*** contain 2000 different type environmental noise that will be useful for data augmentation. The ***'mashups'*** folder contain 3020 test mashup samples for final submission to Kaggle. The ***'test.csv'*** is a mapping from each test sample id to file path present in the ***'mashup'*** folder. The 'sample_submission' is the structure of submission file that must be created for submission.

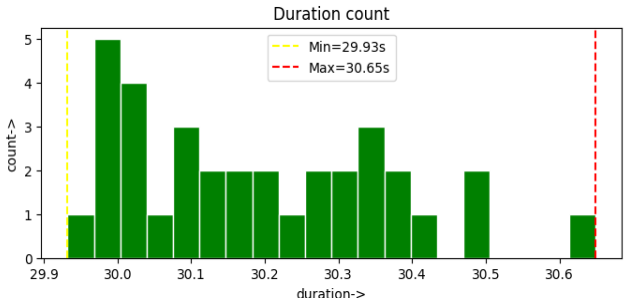
Now the main folder is the ***'genres_stems'*** folder that contains all the stems file for training the classification models. It has 10 folders representing each genre. Each folder contains 100 subfolders. This subfolder is a complete music file that has four stems files such as **'vocals.wav','other.wav','bass.wav'** and **'drums.wav'** in raw **waveform** file format. Example of blue genres given in the image.

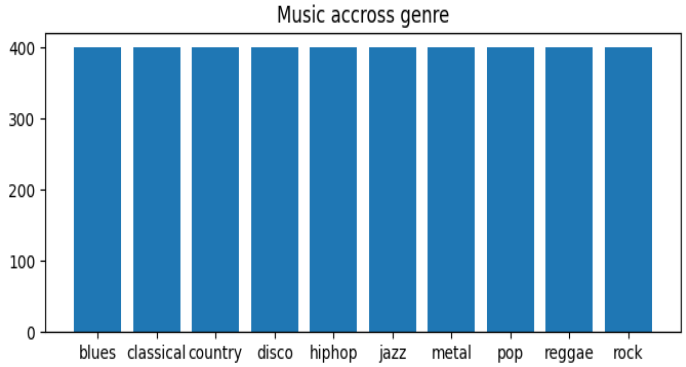


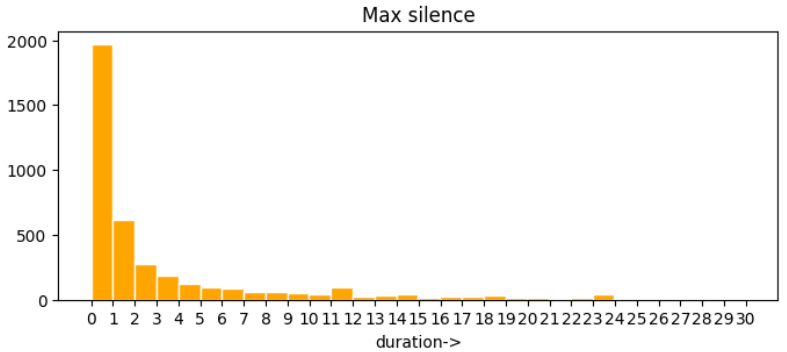


-Dataset Description:

The music dataset are in component separated clean stems file in raw waveform format that can be loaded as a numpy array using librosa or as a tensor using torchaudio. Each stem music is of variable duration which are near 30 second but sample rate of all stems are the same which is 44100 samples/sec. The dataset is balanced as each genre class contains equal amount of music. 100 music so total 400 stems from each genre.

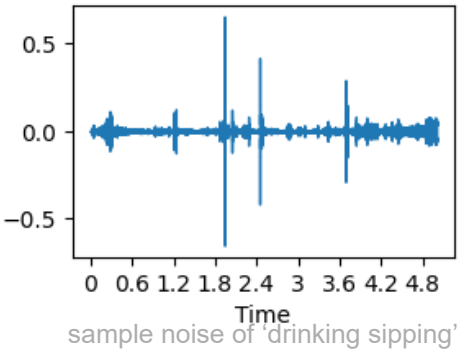




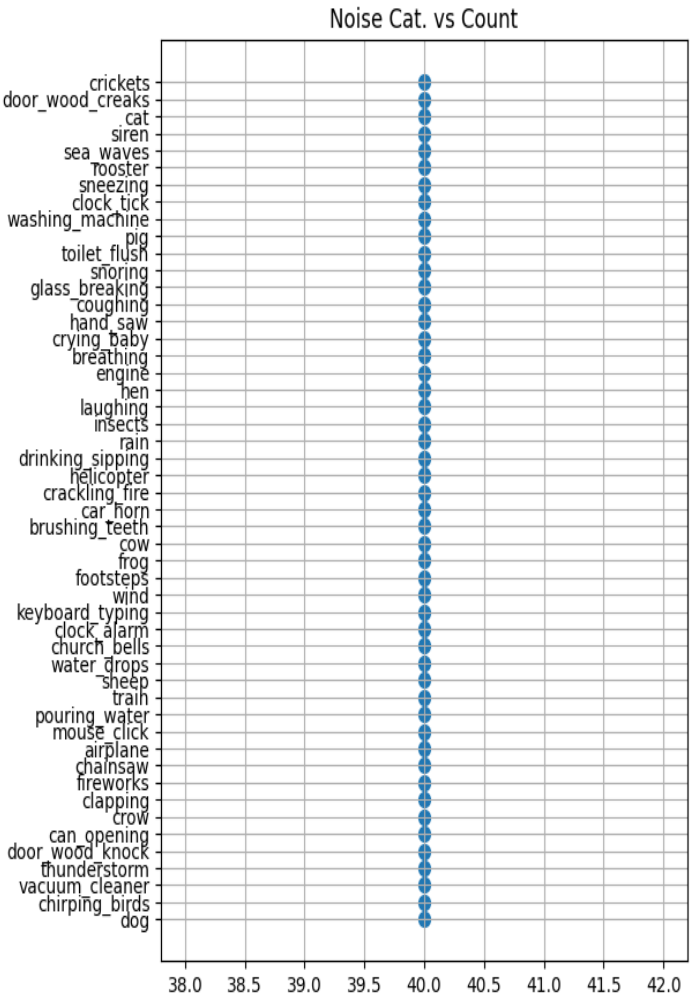


-Noise information:

These are the different categories of noise sound present with the training dataset.



sample noise of 'drinking sipping' category



-Data Augmentation:

I have made three sets of training validation mel-spectrograms as my input data.

1.Dataset 150000 total mel-spectrogram sample:

I have loaded all the stems files as a python dictionary with genre as key and stem as subkey and then I have created some functions that create a noisy mashup within each genre. For each genre I have run a for loop with range upto15000. In each iteration, the loop creates a single noise added mashup sample by selecting four stems file randomly from within the same genre and then adding 4 to 5 noise sounds at random position of the mashup. Thus, creating 15000 noisy mashup per genre and now from this noise added mashup music I have created Mel-Spectrogram for each sample of shape 128*646. The problem with this augmentation is proper stem level train-validation separation does not happen. I have separated 12000 trains and 3000 validations set per genre after creating mel-spectrograms from all 100 stems that ultimately created data leakage in my validation set. Although it does not create any issue in model training it gives misleading validation scores.
After recognizing this problem, I have created another two sets of augmented datasets where I have added some extra synthetic noise and extra augmentation. Those are----

2.Dataset 50000 total train mel-spectrogram and 500 validation mel-spectrogram:

Same process as applied before but with reduced number of samples per genre and with using some extra augmentation function at stem level. Here I have applied augmentation randomly at each stem that is combined to form a single mashup. On each stem I have applied --

1. Random gain
2. Random Pitch shift
3. Random Time stretch and Random Time shift

4. Very small artificial noise with environmental noise from the noise file.

Functions	Purpose
`trim_or_pad(y,sr=SR,duration=DURATION)`	As we saw, stem duration is not equal for all. This function corrects it by padding or trimming to 30 sec for all
`build_dataset(root_dir,test_split=0.20)`	This function loads all stem wav files as numpy arrays.
`load_noise_audios(root_dir,audio_folder,csv_file)`	This loads the noise wav file as a numpy array from environmental noise folders.
`random_time_shift(stem, sr, max_shift_ms=40)`	It takes stem music and computes max_shift from max_shift_ms and randomly takes a number between –max_shift and +max_shift and shifts the stem music.
`create_single_track(stems)`	This takes randomly selected four augmented stems and combines them in a single track.
`aug_stem(stem,sr=SR)`	The main function inside all augmentation happens on a single stem.
`stem_recombination(sl, genre)`	This creates final augmented mashup
`add_noise(mashup,noises,snr_db = 20)`	This function adds randomly selected Environmental noise at random location of 30sec long mashup.

3.Dataset with extra 5000 clean train mel-spectrogram set and 500 clean mel-spectrogram validation set per genre:

After creating this simple only cross stem combined clean dataset, I have added this to the second created dataset resulting in 10000 mix train mashups and 1000 mix validation mashups per genre totaling 100000 mix mel-spectrogram samples for training and 10000 for validation. For this extra 50000 sample creation I did not add any noise or do any augmentation. After creating a second dataset I realize that maybe I have done a little over augmentation and that is why I have added a 50% clean mel sample with the noisy mix.

Mel-spectrogram:

A spectrogram is a two-dimensional visual representation of any sound; one axis captures frequency spectral information, and other axis contains a temporal dimension, and each cell of a 2d diagram contains an amplitude value of a particular frequency at time. I have converted it to a mel scale which is perceptually relevant frequency representation and also huge range of power values compressed to a narrower range.

Functions	Purpose
`Librosa.features.melspectrogram(y=y,sr=SR,n_fft=1024, hop length=1024, n_mels=128)`	Create log-scale Mel-spectrogram from noisy mashup
`librosa.power_to_db(mel, ref=np.max)`	Takes mel-spectrogram And convert to decibel value.

Dataset and Data Loader:

I keep the mel-spectrogram resolution 128 in the spectral axis and 646 in the temporal axis as it will contain enough information and enable faster training process. Those mel-spectrograms are stored in Kaggle Dataset as numpy array format for faster loading during the training process. I use PyTorch Dataset class for organizing my mel-spectrogram dataset. First, I extracted all mel-spectrogram files' paths and stored them in a separate *train_dataset* and *val_dataset* list with their associated label as a list tuple **List[(<path>, <label>), ...]** .

In Dataset definition, a transformer block from torch vision module that first converts the mel-spectrogram from numpy array to torch tensor and then normalizes the mel-spectrogram with a global mean and std of train dataset and for further augmentation in some model training I have added Frequency Masking and Time Masking transformer from torch audio module.

Then I defined two data loader objects, one for training and another for validation for mini batch gradient updates in my training loop with batch size 64 and shuffle is true.

```
train_dataset = MelDataset(
    train_paths,
    data_transformer
)
train_loader = DataLoader(
    train_dataset,
    batch_size=config['batch_size'],
    shuffle=True,
    num_workers=4,
    persistent_workers=True,
    pin_memory=True
)
val_dataset = MelDataset(
    val_paths,
    data_transformer
)
val_loader = DataLoader(
    val_dataset,
    batch_size=config['batch_size'],
    shuffle=True,
    num_workers=4,
    persistent_workers=True,
    pin_memory=True
)
```

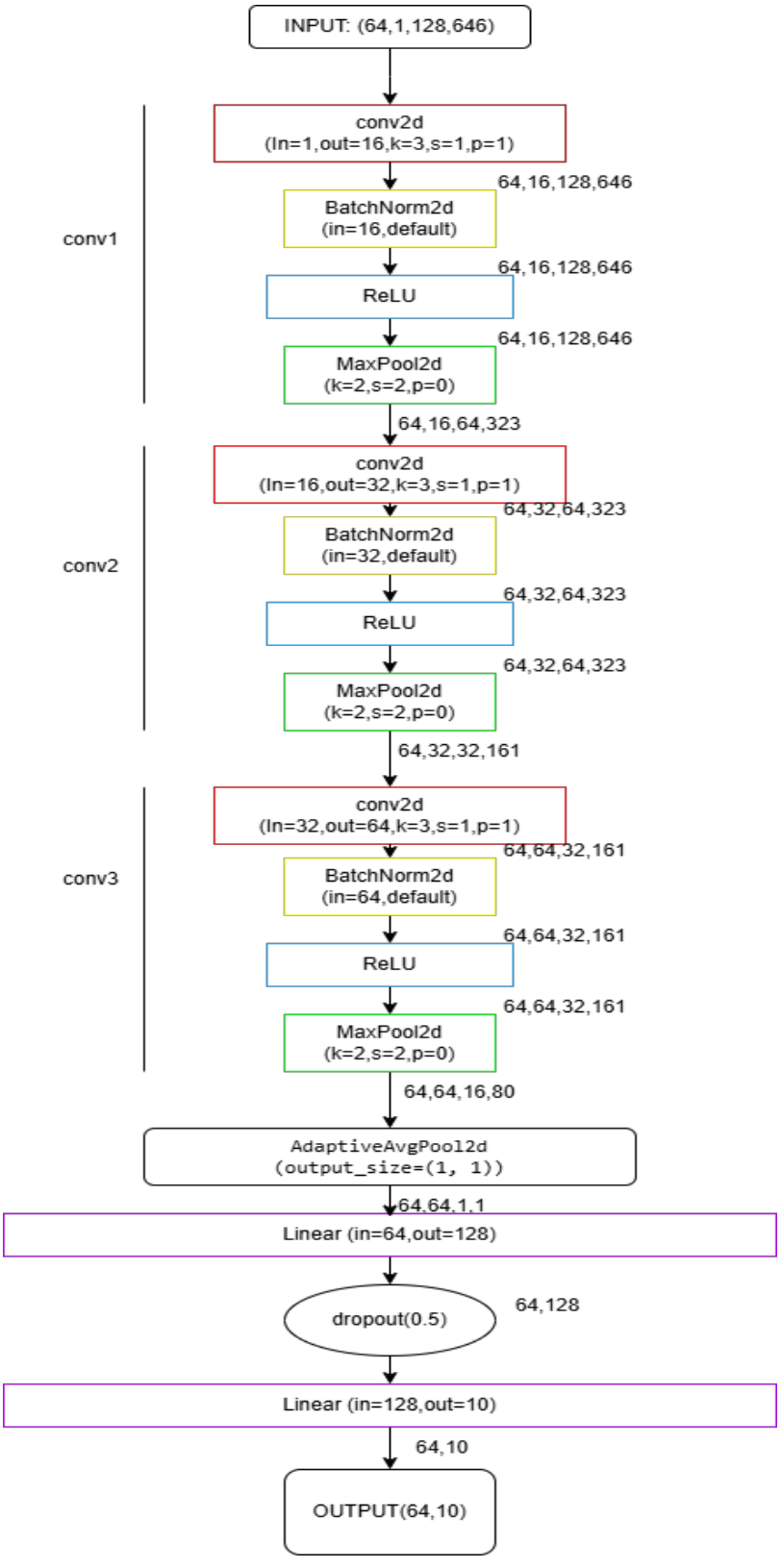
```
class MelDataset(Dataset):
    def __init__(self,paths,transform):  # paths : Lis
        self.paths = paths
        self.transform = transform
    def __len__(self):
        return len(self.paths)
    def __getitem__(self,idx):
        mel = np.load(self.paths[idx][0])
        label = genre_to_idx(self.paths[idx][1])
        #mel = torch.from_numpy(mel).float()
        mel = self.transform(mel)
        label = torch.tensor(label, dtype=torch.long)
        return mel,label

data_transformer = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=[-45.60], std=[16.60]),
    T.FrequencyMasking(freq_mask_param=15),
    T.TimeMasking(time_mask_param=40)
])
```

Modeling & Experimentation

For this project, I have built three models 1. ***CNN from scratch***, 2. ***CLSTM from scratch*** and 3. ***Pretrained ResNet18 with finetune.***

Model-1: CNN

Architecture diagram:



Salient Points:

As mel-spectrogram is image like data in 2d space with one channel so I pick CNN as my first model and then apply three convolution layers with small kernel size 3*3 and increased filter count 16->32->64 with decreased spatial dimensions. Small kernel size enables reduced parameter counts and as music rhythm changes within a very small fraction of time so keeping kernel size small captures that info effectively and padding of 1 also preserves spatial info at borders of my input.

I use max pooling to reduce spatial size of feature maps and adaptive global average pooling to further reduce parameter count to linear scale.

Dropout of 0.5 in linear layers to let the model not depend on specific weights and reduce the overfitting.

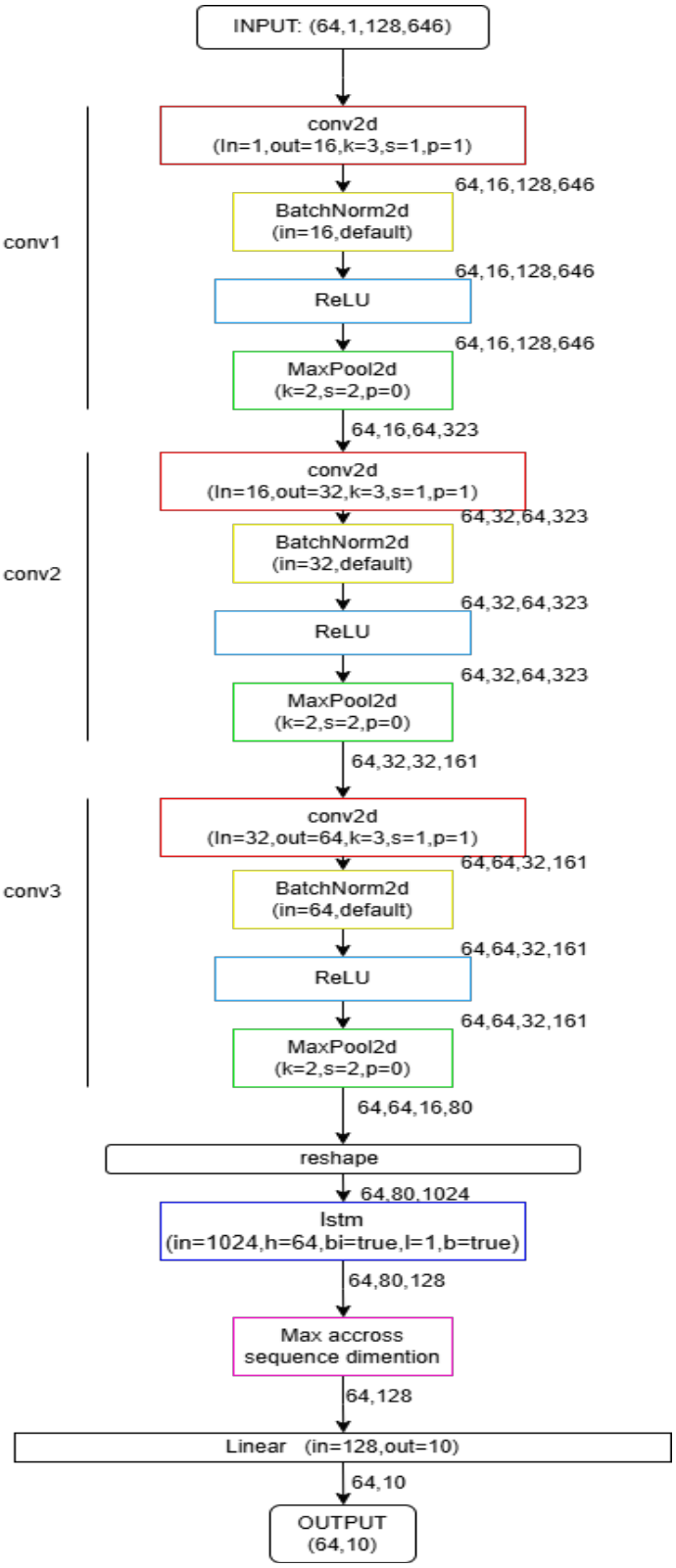
Batch normalization is used to make the values at the same scale for better stabilization and faster convergence of the learning process and ReLU as a non-linear activation function.

Cross entropy loss as standard for classification and it also penalizes incorrect prediction heavily. It takes logits from model output and computes SoftMax probability inside it then computes loss.

Adam as optimizer with a low learning rate of 0.0001.

Model-2: CLSTM

Architecture Diagram:
(with shape flow through layers)



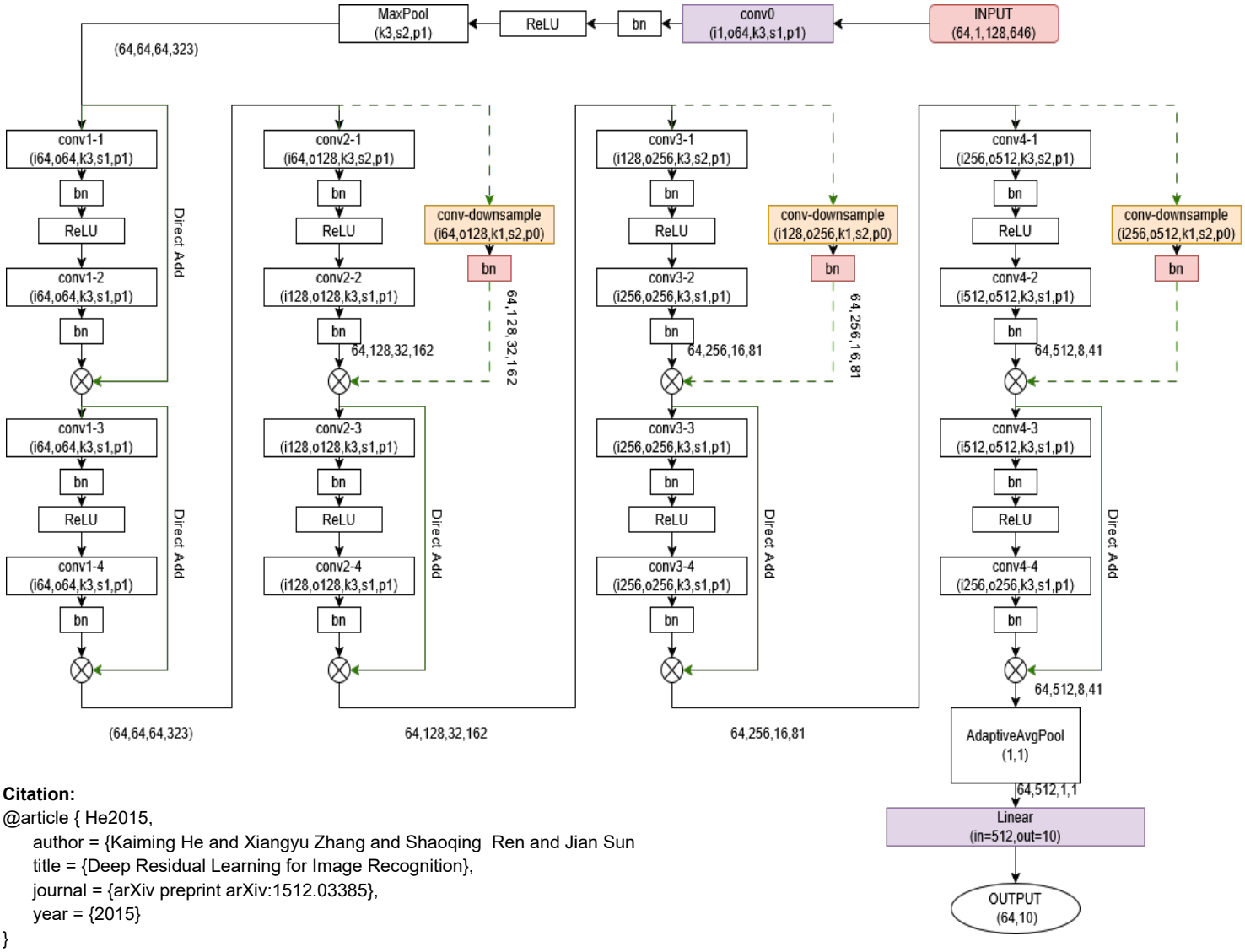
Salient Points:

I have used a sequence-based model as music is a sequence in the temporal domain, and the sequence-based model captures the spectral change with amplitude value through time and based on that it classifies the correct genre.

I have used the same CNN blocks from the previous model and added a bidirectional lstm layer just after the third convolution block. The same mel-spectrogram is used but now as a sequence of the time axis and frequency dimensions will be as input vector for the lstm block. Then I applied max pooling along the temporal axis. At last the linear layer takes the pooled hidden states and gives the logit.

Model-3: Pre-trained ResNet18

Architecture diagram:



Citation:

@article { He2015,
author = {Kaiming He and Xiangyu Zhang and Shaoqing Ren and Jian Sun
title = {Deep Residual Learning for Image Recognition},
journal = {arXiv preprint arXiv:1512.03385},
year = {2015}
}

I have used a pre-trained model from the torch vision pretrained models' section. I have replaced the first convolution layer with my custom convolution layer. And the classification head with my custom classification head and then I have trained the whole model without freezing its parameters.

Replacement with these layers:

1. `res_net.conv1 = nn.Conv2d(1,64,kernel_size=(3, 3), stride=1, padding=1, bias=False)`

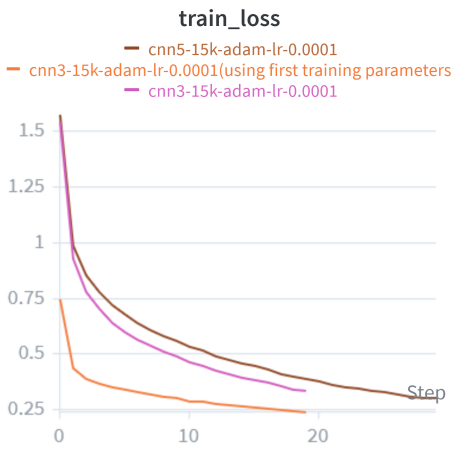
2. `res_net.fc= nn.Linear(res_net.fc.in_features,10)`

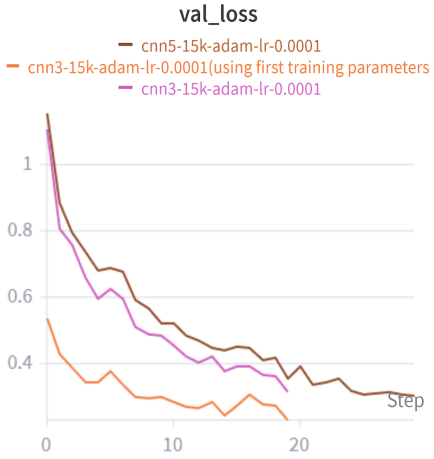
Performance and Comparative Analysis:

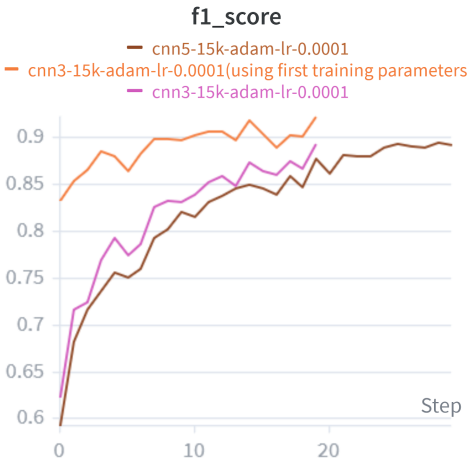
Macro F1 score is the primary evolution metric used to compare the model performance.For other two dataset I have used accuracy score also.

1.With 150000 mel-spectrogram dataset:-

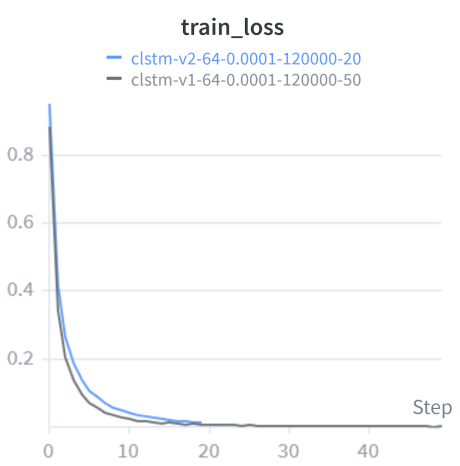
CNN: with 120000 training and 30000 validation sample I have trained 3 version of CNN

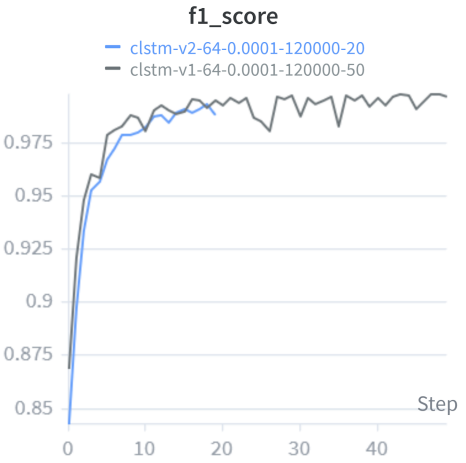




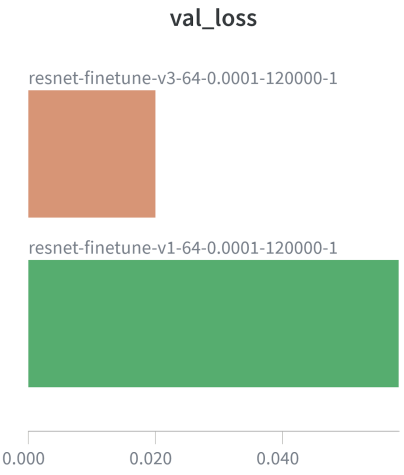


CLSTM : with this dataset I have trained two version of CLSTM

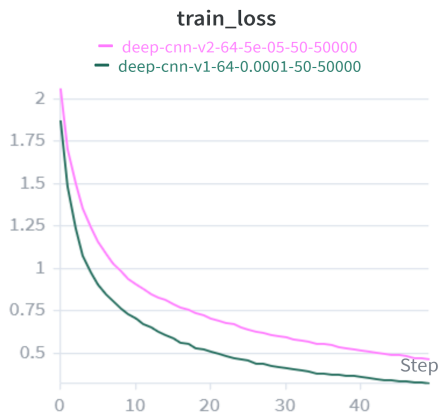
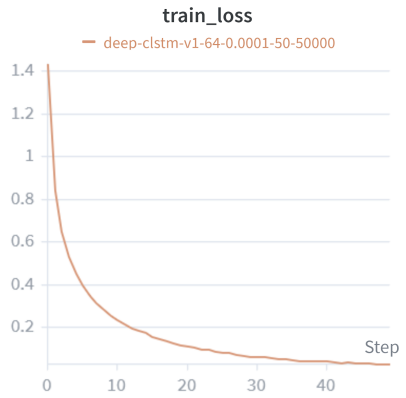
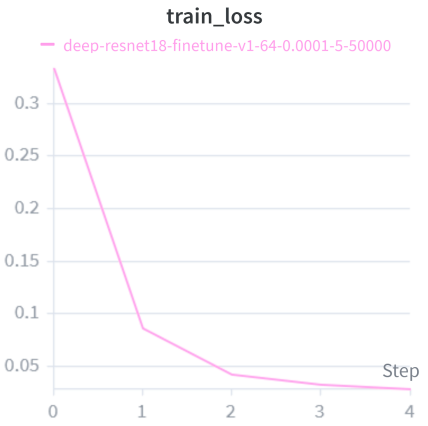
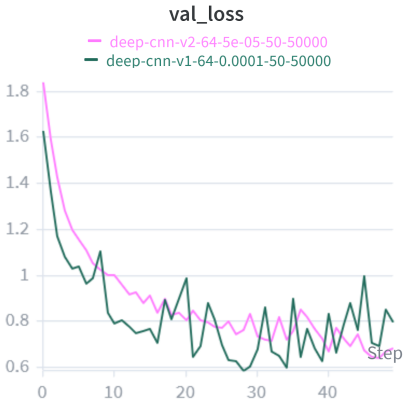
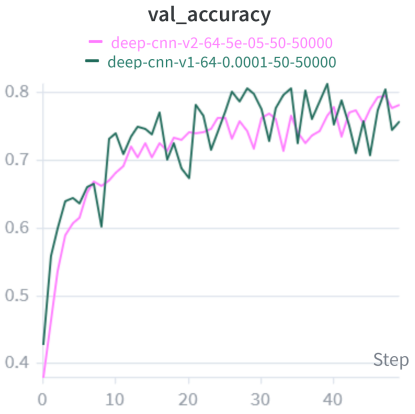
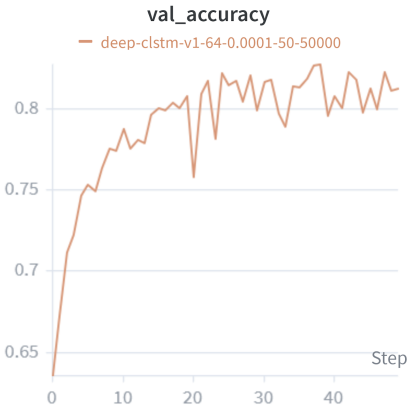
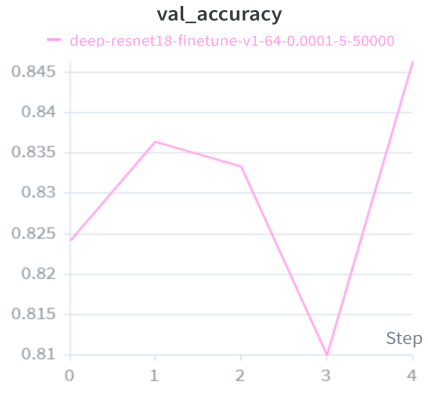
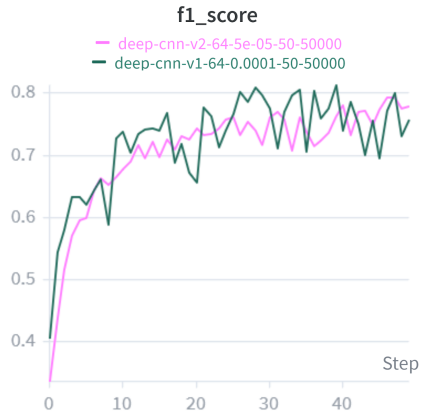
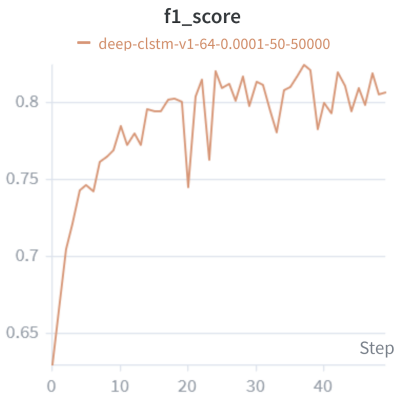
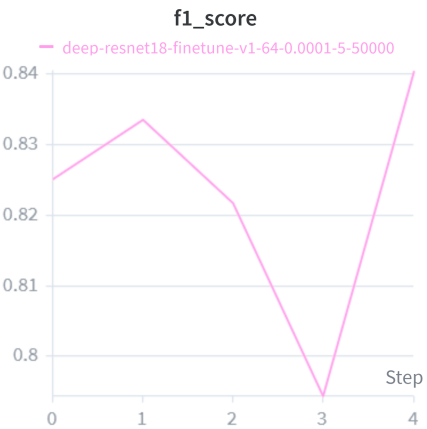




ResNet18:

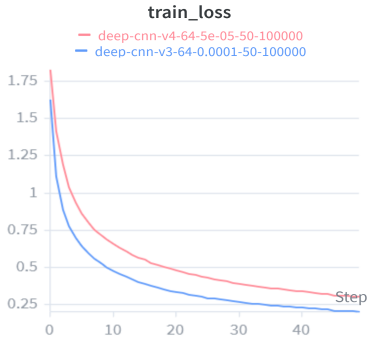


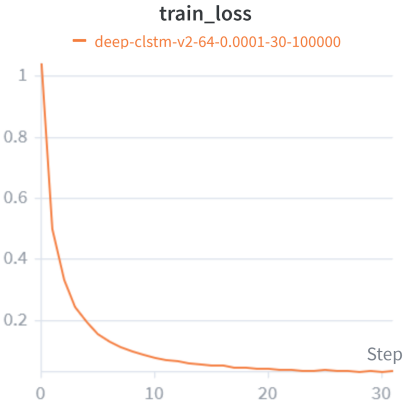
2.With 50000+5000 dataset: with 50000 as training and 5000 as validation sample

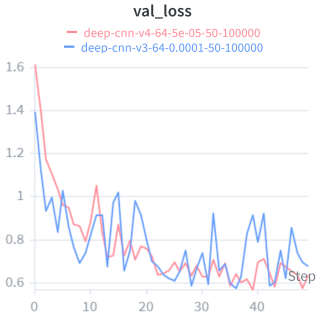
CNN	CLSTM	ResNet 18
		
		
		
		

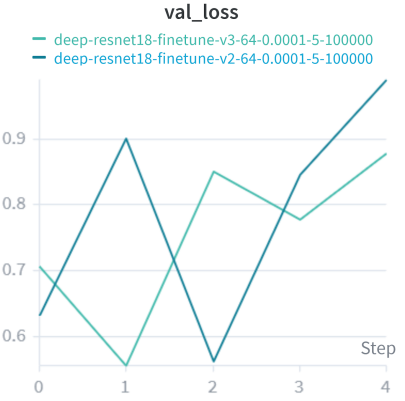
3.With 50000noisy+50000clean dataset: With 100000 as train and 10000 as validation sample

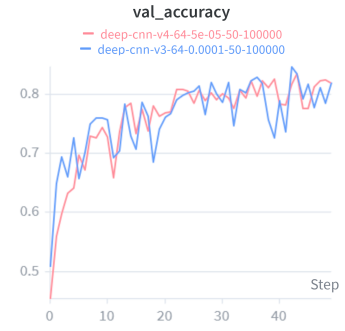
CNN **CLSTM** **ResNet18**

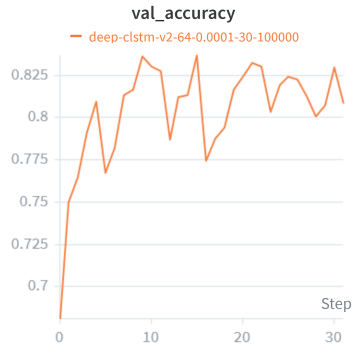


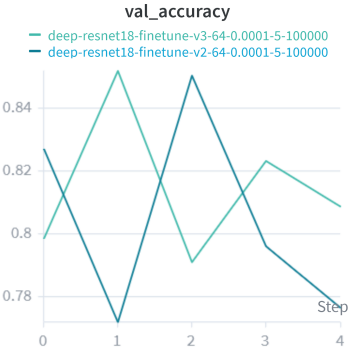


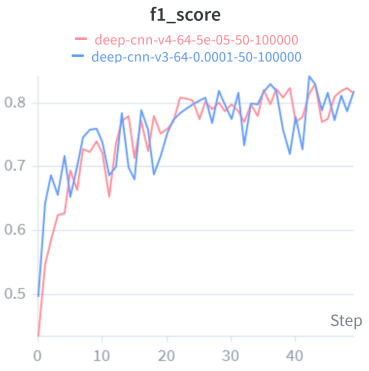


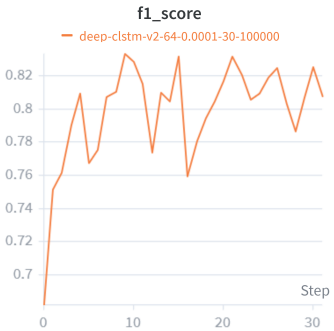


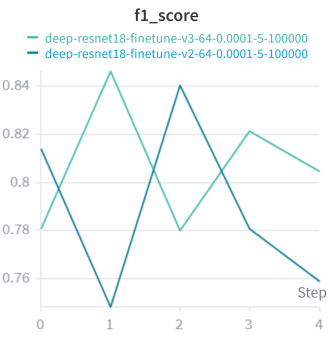












Comparative Table:

1.Trained with first 150000 dataset-

Model name	epoch	lr	Test f1_score
CNN	20	0.0001	0.71
Retrain CNN	20	0.0001	0.80
CNN with time and frequency masking	30	0.0001	0.75
CLSTM	50	0.0001	0.78
CLSTM with time and frequency masking	20	0.0001	0.82
ResNet18 with extra added linear layer	1	0.0001	0.87
Default ResNet18	1	0.0001	0.90

2.Trained with second 50000 noisy dataset-

Model name	epoch	lr	Test f1_score
CNN	50	0.0001	0.54
CNN	50	0.00005	0.66
CLSTM	50	0.0001	0.73
ResNet	5	0.0001	0.82

3. Trained with 100000 mix dataset-

Model name	epoch	lr	Test f1_score
CNN	50	0.0001	0.73
CNN	50	0.00005	0.72
CLSTM	30	0.0001	0.80
ResNet18	5	0.0001	0.86
ResNet18 with extra linear layer	5	0.0001	0.85

Conclusion:

At first I mistakenly mixed the train and validation set. I did not properly separate the train and validation set at stem level.Although that does not affect the training process, validation got overly perfect and misleading. The only way to know my model performance was to submit and get the test score on Kaggle. Though that 150000 dataset performs well on these three models that I have trained. Later I have created two datasets with proper stem separation but those datasets did not perform well, maybe because of a smaller number of samples or maybe because of over augmentation that reduces actual music features of my dataset.
The major challenge was to create and augment the training dataset so that it works for noisy mashup.

ResNet18 and Clstm perform well so I am thinking of combining ResNet18 with Lstm that has potential to improve my test f1 score.

References:

1.Pytorch : torch — PyTorch 2.11 documentation
2.torchaudio for audio data :torchaudio — Torchaudio 2.10.0 documentation
3.huggingface audio project understanding : Welcome to the Hugging Face Audio course!
4.Resnet18: ResNet — Torchvision 0.25 documentation
5.ResNet model architecture image(Original Paper) : ResNet paper
5.librosa for audio features :Core IO and DSP — librosa 0.11.0 documentation